Bài ôn tập chương 5

1. Dòng đầu tiên của định nghĩa/xây dựng hàm được gọi là tiêu đề hàm, bao gồm

các thông tin gì?

- **Từ khóa khai báo (Keyword):** Cho máy tính biết bạn đang bắt đầu định nghĩa một hàm
- **Tên hàm (Function Name):** Là định danh dùng để gọi hàm sau này.
- **Danh sách tham số (Parameters):** Nằm trong dấu ngoặc đơn `()`. Đây là các biến đầu vào mà hàm cần để xử lý. Nếu hàm không cần đầu vào, ngoặc đơn sẽ để trống.
- **Kiểu dữ liệu trả về (Return Type):** Cho biết kết quả đầu ra của hàm thuộc loại gì (số nguyên, chuỗi, logic...).

2. Cách xây dựng và sử dụng hàm không có giá trị trả về?

- Hàm không trả về giá trị được gọi là **hàm void**. Đây là loại hàm thực hiện một nhiệm vụ cụ thể (như in dữ liệu, sửa đổi biến toàn cục, hoặc vẽ hình) mà không gửi lại kết quả tính toán nào cho nơi gọi nó.

- Vì hàm không trả về giá trị, **không thể** gán nó cho một biến hoặc dùng nó trong các phép toán (như `int x = tenHam();` là sai). Chỉ cần gọi tên hàm và truyền tham số (nếu có).

3. Cách xây dựng và sử dụng hàm có giá trị trả về?

- Hàm **có giá trị trả về** giống như một "nhà máy chế biến", đưa nguyên liệu vào (tham số), và nó phải gửi lại một thành phẩm (kết quả) cho bạn để bạn có thể tiếp tục sử dụng.

- Vì hàm này "trả về" một giá trị, bạn có 3 cách phổ biến để sử dụng kết quả đó:

+ Cách 1: Gán vào một biến

+ Cách 2: Dùng trực tiếp trong biểu thức

+ Cách 3: Dùng làm tham số cho hàm khác

4. Sự khác nhau giữa định nghĩa hàm và nguyên mẫu hàm?

3. Bảng so sánh nhanh

Đặc điểm	Nguyên mẫu hàm (Prototype)	Định nghĩa hàm (Definition)
Dấu kết thúc	Có dấu `;`	Không có dấu `;`, dùng `{ }`
Nội dung	Chỉ có tên, kiểu trả về, tham số	Có đầy đủ logic/code bên trong
Số lần xuất hiện	Có thể khai báo nhiều lần	**Duy nhất** 1 lần trong chương trình
Vai trò	"Giới thiệu" hàm với trình biên dịch	"Thực thi" công việc của hàm

5. Biến cục bộ là gì? điểm đặc biệt của nó?

Biến cục bộ là biến được khai báo bên trong một **hàm** hoặc một **khối lệnh** (thường nằm giữa cặp ngoặc nhọn `{ }`).

Biến cục bộ có 4 đặc tính quan trọng mà bạn cần nhớ để tránh bị "bug":

- **Phạm vi (Scope) hạn hẹp:** Bạn chỉ có thể truy cập và sử dụng biến đó bên trong hàm/khối lệnh đã khai báo nó. Nếu bạn cố gọi nó từ bên ngoài, trình biên dịch sẽ báo lỗi ngay lập tức vì nó "không biết đứa này là ai".
- **Vòng đời (Lifetime) ngắn ngủi:** Biến cục bộ được sinh ra khi hàm bắt đầu chạy và sẽ bị "tiêu hủy" (giải phóng bộ nhớ) ngay khi hàm kết thúc.
- **Vùng nhớ Stack:** Trong các ngôn ngữ như C++, biến cục bộ thường được lưu trữ trong vùng nhớ **Stack**. Đây là vùng nhớ có tốc độ truy xuất cực nhanh nhưng dung lượng có hạn.
- **Hiện tượng "Shadowing" (Che lấp):** Nếu bạn đặt tên một biến cục bộ trùng với tên một biến toàn cục, chương trình sẽ ưu tiên dùng biến cục bộ bên trong hàm đó. Biến toàn cục lúc này sẽ bị "che khuất" tạm thời.

6. Biến toàn cục là gì? điểm đặc biệt của nó?

Biến toàn cục là biến được khai báo bên ngoài tất cả các hàm, các khối lệnh (thường nằm ở ngay đầu file code, dưới các dòng `#include`).

Vì nó không thuộc về riêng một "nhà" (hàm) nào, nên tất cả các hàm trong chương trình đều có quyền truy cập và sử dụng nó.

Biến toàn cục có những đặc tính "đặc biệt" nhưng cũng đầy rủi ro:

- **Phạm vi (Scope) "phủ sóng toàn tập":** Có tác dụng từ điểm khai báo cho đến hết file chương trình. Bất kỳ hàm nào nằm dưới nó đều có thể "gọi tên" nó.
- **Vòng đời (Lifetime) "trường thọ":** Nó được sinh ra ngay khi chương trình bắt đầu chạy và chỉ kết thúc khi bạn tắt chương trình. Nó không bị xóa đi sau khi một hàm kết thúc như biến cục bộ.
- **Tự động khởi tạo:** Nếu bạn không gán giá trị cho biến toàn cục, trình biên dịch sẽ tự động gán cho nó giá trị mặc định (thường là `0` đối với kiểu số, hoặc `NULL` đối với con trỏ).
- **Vùng nhớ Data Segment:** Khác với biến cục bộ nằm ở Stack, biến toàn cục được lưu trữ ở một vùng nhớ riêng gọi là **Data Segment** (vùng nhớ tĩnh).

7. Hàm main() nên đặt ở đâu?

Cách 1: Đặt ở cuối file. Đây là cách "cổ điển". Bạn định nghĩa toàn bộ các hàm con ở bên trên, sau đó mới viết hàm `main()`. Cách này giúp bạn không cần dùng đến "nguyên mẫu hàm" (prototype) vì mọi thứ đã được khai báo trước khi `main()` gọi chúng.

Cách 2: Đặt ở đầu file (Khuyên dùng). Bạn đặt hàm `main()` ngay sau các dòng `#include` và danh sách các **nguyên mẫu hàm**. Cách này giúp người đọc code (hoặc chính bạn sau này) vừa mở file ra là thấy ngay "luồng chính" của chương trình mà không phải kéo xuống tận đáy.

8. Tham số của hàm là gì?

Tham số của hàm là những biến được khai báo trong dấu ngoặc đơn `( )` khi chúng ta định nghĩa hàm. Chúng đóng vai trò là các "cổng tiếp nhận" dữ liệu từ bên ngoài truyền vào để hàm có thể xử lý.

9. Tham số hình thức là gì? Tham số thực là gì?

Tham số hình thức (Formal Parameters): Là các biến được khai báo ở **phần định nghĩa hàm**. Chúng chỉ là những "kẻ đóng thế" (placeholders), chưa có giá trị thực sự cho đến khi hàm được gọi.

Tham số thực (Actual Parameters/Arguments): Là giá trị thực sự hoặc biến cụ thể mà bạn truyền vào khi **gọi hàm** trong `main()` hoặc trong một hàm khác.

10.Có mấy hình thức truyền tham số? Điểm khác nhau giữa các hình thức này?

Có 4 hình thức:

Đặc điểm	Truyền tham trị (Pass by Value)	Truyền tham chiếu (Pass by Reference)	Truyền địa chỉ (Pass by Address/Pointer)
Cú pháp	void ham(int x)	void ham(int &x)	void ham(int *x)
Cơ chế	Tạo ra một bản sao của dữ liệu.	Dùng chung bí danh (alias) của biến gốc.	Truyền địa chỉ ô nhớ của biến gốc.
Thay đổi	Thay đổi trong hàm không ảnh hưởng biến gốc.	Thay đổi trong hàm có làm thay đổi biến gốc.	Thay đổi giá trị tại vùng nhớ có làm thay đổi biến gốc.
Bộ nhớ	Tốn bộ nhớ để tạo bản sao (không tốt cho dữ liệu lớn).	Tiết kiệm bộ nhớ vì không tạo bản sao.	Tiết kiệm bộ nhớ, nhưng cú pháp hơi phức tạp.
Độ an toàn	Cao nhất (biến gốc được bảo vệ).	Thấp hơn (biến gốc có thể bị sửa đổi ngoài ý muốn).	Thấp (có nguy cơ lỗi con trỏ/vùng nhớ).

11. Làm 10 câu hỏi trắc nghiệm trong tài liệu lý thuyết trang 60 – 63.

12.Viết hàm tính tích của 2 số a, b theo dạng sau:

int TinhTich(int a, int b)

Áp dụng hàm tính tích trên để viết chương trình tính tích của 5 số a, b, c, d, e.

13.Viết hàm để kiểm tra 1 số có phải là số chẵn hay không theo dạng sau:

int LaSoChan(int a)

Áp dụng hàm trên để viết chương trình cho nhập vào 1 số nguyên N. Nhập vào N số nguyên, in ra các số chẵn trong các số nguyên vừa nhập.

14. Viết chương trình thực hiện các công việc sau:

a. Viết hàm để kiểm tra 1 số a có phải là ước số của b không theo định dạng sau:

int LaUocSo (int a, int b)

b. Viết hàm đếm số ước số của số nguyên N theo định dạng sau:

int DemUocSo(int N)

c. Áp dụng hàm DemUocSo để viết hàm kiểm tra 1 số có phải là số nguyên tố hay không (số nguyên tố là số chỉ có 2 ước số) theo dạng sau:

int LaSoNT(int a)

d. In ra các số nguyên tố <100.

e. Nhập vào số nguyên N, in ra N số nguyên tố đầu tiên.